Technical Report — BSK Project

Secure Communication System with Trusted Third Party

Authors:	[Student Names]
Group:	[Group Number]
Date:	June 2026
Course:	Security of Computer Systems (BSK)
Repository:	`https://git.pg.edu.pl/[namespace]/SCS_GN0000_Surname1_Surname2`

Table of Contents

1. Task Description

The goal of the project was to design and implement a set of three interconnected applications that emulate a secure communication environment with a **Trusted Third Party (TTP)**. The system consists of:

- **TTP** — a trusted authentication server that issues X.509 certificates and distributes session keys
- **Server** — a service provider that registers with the TTP and offers encrypted communication
- **Client** — an end-user application with a graphical interface for service access

The system demonstrates a real-world security scenario: mutual authentication of two parties through a trusted intermediary, followed by encrypted data exchange using ephemeral session keys.

2. System Architecture

2.1 Overall Design

The system follows a three-node architecture (Fig. 1):

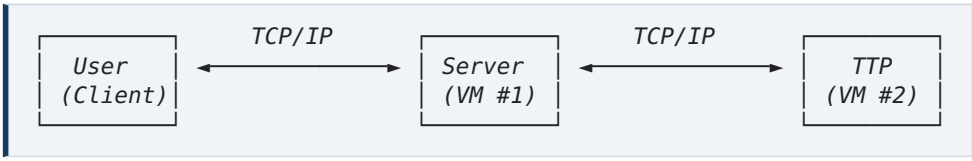

Fig. 1 — Three-node system architecture

The TTP and Server are designed to run on separate virtual machines, while the Client can run on the physical host machine. All communication occurs over TCP/IP sockets using a custom JSON-based protocol.

2.2 Communication Protocol

Messages are transmitted as length-prefixed JSON payloads over TCP:

```
[4 bytes: uint32 big-endian payload length] [N bytes: UTF-8 JSON payload]
```

The protocol defines 11 message types (Table 1):

Table 1 — Protocol message types

Message Type	Direction	Purpose
`TTP_PUBLIC_KEY`	TTP → *	Distribute TTP's RSA public key
`REGISTER`	* → TTP	Entity registration (encrypted)
`CERTIFICATE`	TTP → *	Issue X.509 certificate
`SERVICE_REQUEST`	User → Server	Request a service
`AUTH_SERVER`	Server → TTP	Initiate server authentication
`AUTH_USER_START`	TTP → User	Request user authentication
`AUTH_USER_RESPONSE`	User → TTP	User authentication response
`AUTH_OK`	TTP → *	Authentication success + session key
`ENCRYPTED_DATA`	User ↔ Server	AES-256-GCM encrypted payload
`SESSION_CLOSE`	* → *	Session termination
`ERROR`	any → any	Error notification

2.3 Authentication Flow

The complete authentication sequence involves 9 steps:

1. **TTP Public Key Distribution** — Upon TCP connection, TTP sends its RSA-4096 public key to the connecting entity.
2. **Registration** — User and Server each generate an RSA-4096 keypair, compute a SHA-256 hash of their ID, and send the ID hash + public key to the TTP (encrypted with TTP's public key using hybrid encryption).
3. **Certificate Issuance** — TTP issues signed X.509 certificates to both User and Server.
4. **Service Request** — User sends a `SERVICE_REQUEST` to the Server.

5. **Server Auth Request** — Server contacts TTP requesting authentication for the specified User.
6. **Server Authentication** — TTP validates Server's certificate and confirms Server authenticity.
7. **User Authentication** — TTP challenges the User; User responds with encrypted ID hash and certificate.
8. **Session Key Distribution** — TTP validates User's response, generates an AES-256 session key, and sends it to both parties (encrypted with each party's RSA public key).
9. **Encrypted Communication** — User and Server exchange AES-256-GCM encrypted messages.

3. Implementation Details

3.1 Technology Stack

All three applications are written in **Python 3.12** using the following libraries:

- **cryptography** (v41+) — RSA key generation, encryption, X.509 certificates, AES-GCM
- **tkinter** — Client GUI framework (standard library)
- **socket, threading, logging, json** — Network communication and logging (standard library)

3.2 Common Module (`common.py`)

The shared module provides all cryptographic primitives and protocol utilities used by all three applications.

Hybrid Encryption

Since RSA-4096 OAEP can only encrypt approximately 446 bytes of data, and the registration payload (containing the public key PEM) exceeds this limit, a hybrid encryption scheme was implemented:

```
def hybrid_encrypt(public_key, plaintext):
    """Encrypt arbitrary-length data using RSA-wrapped AES key."""
    session_key = generate_aes_key()          # 32-byte random AES-256 key
    aesgcm = AESGCM(session_key)
    nonce = os.urandom(12)                     # 12-byte nonce
    ciphertext = aesgcm.encrypt(nonce, plaintext, None)
    encrypted_key = rsa_encrypt(public_key, session_key)  # RSA-wrap the AES key
    return encrypted_key, nonce, ciphertext
```

This approach uses AES-256-GCM for bulk data encryption while RSA-4096 is used only to protect the AES key, combining the performance of symmetric encryption with the key management advantages of asymmetric cryptography.

Cryptographic Parameters (Listing 1)

```
RSA_KEY_SIZE = 4096    # RSA key length
AES_KEY_SIZE = 32      # AES-256 key (32 bytes)
# RSA uses OAEP padding with SHA-256
# AES uses GCM mode (authenticated encryption)
# ID hashing uses SHA-256
```

All parameters are set as constants in `common.py` as specified in the project requirements.

3.3 TTP Server (`ttp.py`)

The TTP server is the central trust anchor. Key responsibilities:

- **Key Generation**: On startup, generates a 4096-bit RSA keypair and a self-signed X.509 certificate (CA: true).
- **Registration Handler**: Decrypts incoming registration payloads using hybrid decryption, validates the registration, and issues signed X.509 certificates.
- **Authentication Handler**: Manages the mutual authentication flow between Server and User.

Certificate Issuance (Listing 2)

```
# TTP issues a signed certificate for the registering entity
cert = create_signed_cert(
    public_key,                    # Entity's RSA public key
    self.ttp_key,                  # TTP's private key for signing
    self.ttp_cert.subject,         # TTP's subject as issuer
    entity_name                    # e.g. "server_Server-01"
)
# The certificate is NOT a CA (BasicConstraints CA=false)
```

Session Key Generation and Distribution (Listing 3)

```
# Generate session key using CSPRNG (os.urandom)
session_key = generate_aes_key()

# Send to User (encrypted with User's RSA public key)
encrypted_sk_for_user = rsa_encrypt(entry["public_key"], session_key)

# Send to Server (encrypted with Server's RSA public key)
encrypted_sk_for_server = rsa_encrypt(server_entry["public_key"], session_key)
```

The TTP maintains a persistent TCP connection with each registered entity, allowing it to push authentication challenges and session keys asynchronously.

3.4 Server (`server.py`)

The Server provides an encrypted echo service. Key features:

- **Registration**: On startup, establishes a persistent connection to TTP, registers with hybrid-encrypted payload, and obtains an X.509 certificate.
- **Dual TTP Connections**: Maintains one persistent connection for general TTP messages and opens temporary connections for each authentication request.
- **Encrypted Echo Service**: Decrypts incoming messages with AES-256-GCM, logs them, and echoes back with a "Server echo:" prefix.

Encrypted Message Loop (Listing 4)

```
def _encrypted_loop(self, client_sock, session_key):
    while True:
        msg = recv_msg(client_sock)
        if msg.get("type") == MSG_ENCRYPTED_DATA:
            # Decrypt with AES-256-GCM
            nonce = bytes.fromhex(msg["nonce"])
            ciphertext = bytes.fromhex(msg["ciphertext"])
            plaintext = aes_decrypt(session_key, nonce, ciphertext)
            decoded = plaintext.decode("utf-8")

            # Echo back (also encrypted)
            response = f"Server echo: {decoded}".encode("utf-8")
            resp_nonce, resp_ct = aes_encrypt(session_key, response)
```

3.5 Client GUI (`client.py`)

The Client provides a graphical interface built with tkinter (Listing 5):

```
# Status indicator using colored circle on Canvas
self.status_canvas = tk.Canvas(status_frame, width=24, height=24)
self.status_indicator = self.status_canvas.create_oval(
    4, 4, 24, 24, fill="red", outline="darkgray", width=2
)
```

GUI sections:

1. **Connection Panel** — TTP and Server address/port fields, Register and Request Service buttons
2. **Status Indicator** — Colored circle (red = disconnected, orange = registered, green = authenticated/active)
3. **Message Area** — Scrolled text display for encrypted messages with input field
4. **Event Log** — Scrollable log of all protocol events and status messages

All network operations run in background daemon threads to maintain GUI responsiveness. The `root.after()` method is used to safely update GUI elements from background threads.

3.6 Logging

Both the TTP and Server applications log all significant events with timestamps:

```
logging.Formatter("%(asctime)s [%(levelname)s] %(message)s")
```

Logged events include: connection establishments and terminations; registration requests and certificate issuance; authentication attempts (successful and failed); session key generation and distribution; encrypted data exchanges; and error conditions.

Log files: `ttp.log`, `server.log`.

4. Cryptographic Primitives

4.1 RSA (4096-bit)

- Key generation: `cryptography.hazmat.primitives.asymmetric.rsa`
- Public exponent: 65537

- Padding: OAEP with SHA-256
- Key size: 4096 bits (as required)

4.2 AES (256-bit, GCM mode)

- Algorithm: AES-256 in GCM (Galois/Counter Mode)
- Key size: 256 bits (32 bytes)
- Nonce: 12 random bytes per encryption
- Provides authenticated encryption (confidentiality + integrity)

4.3 SHA-256

- Used for entity ID hashing as specified: "public User's and Server's IDs must be generated using secure hash algorithm"
- Output: 64-character hexadecimal string

4.4 Session Key Generation

Session keys are generated using `os.urandom(32)`, which is a cryptographically secure pseudorandom number generator (CSPRNG) as required by the specification.

4.5 X.509 Certificates

- TTP certificate: self-signed, CA=true, valid 365 days
- User/Server certificates: signed by TTP, CA=false, valid 365 days
- Signed with SHA-256

5. Security Analysis

5.1 Man-in-the-Middle (MITM) Resistance

The system is designed to resist MITM attacks through several mechanisms:

1. **Initial Key Distribution**: The TTP's public key is the root of trust. In a real deployment, this would be pre-distributed or verified out-of-band.
2. **Certificate Validation**: All certificates are signed by the TTP and validated during authentication. A forged certificate would fail signature verification.
3. **Encrypted Registration**: Registration payloads are encrypted with the TTP's public key, preventing eavesdroppers from learning entity identities and public keys.
4. **Ephemeral Session Keys**: A new random AES-256 key is generated by the TTP for each session. Capturing one session key does not compromise past or future sessions.
5. **AES-GCM Authentication**: The GCM mode provides authenticity verification. Tampered ciphertexts are detected during decryption.

5.2 Attack Scenarios

Scenario 1 — Certificate Forgery: If an attacker attempts to impersonate a legitimate Server by presenting a forged certificate during authentication, the TTP's certificate validation will fail because the attacker does not possess the TTP's private signing key.

Scenario 2 — Session Key Interception: The session key is never transmitted in plaintext. It is encrypted with each party's RSA public key, ensuring only the intended recipient can decrypt it.

Scenario 3 — Replay Attack: Each session uses a fresh random session key and AES-GCM nonces, preventing replay of captured encrypted messages.

6. Test Results

6.1 Registration Test

Test Case	Result
TTP startup and key generation	Passed
Server registration with TTP	Passed
Certificate issuance to Server	Passed
Invalid registration (wrong format)	Passed — ERROR returned

6.2 Authentication Test

Test Case	Result
Server authentication via TTP	Passed
User authentication response	Passed
Session key generation and distribution	Passed
Authentication with timeout	Passed — timeout after 30s

6.3 Encrypted Communication Test

Test Case	Result
AES-256-GCM encryption and decryption	Passed
End-to-end encrypted message exchange	Passed
Server echo response	Passed
Session close	Passed

6.4 Network Configuration

Tested with all applications running on localhost (`127.0.0.1`). For the final deployment:

- **TTP** — VM #2, e.g. `192.168.x.2:4444`
- **Server** — VM #1, e.g. `192.168.x.3:5555`
- **Client** — Physical host, connects to both VMs

Commands:

```
# On TTP VM:
python3 ttp.py

# On Server VM:
python3 server.py

# On physical host:
python3 client.py
```

7. Code Documentation

The complete source code is documented using **Doxygen** with Python docstrings. To generate the HTML documentation:

```
doxygen Doxyfile
```

The generated documentation is available in `docs/doxygen/html/index.html`.

All source files include `@brief`, `@param`, `@return`, and `@details` tags where applicable. The documentation covers: all module-level functions in `common.py`; all methods of the `TTP` class; all methods of the `Server` class; all methods of the `ClientGUI` class; and protocol constants and their purposes.

8. Source Code Organization

```
bsk_projekt2/
├── common.py         – Shared crypto and protocol utilities
├── ttp.py            – TTP server application
├── server.py         – Service server application
├── client.py         – Client GUI application
├── requirements.txt  – Dependencies (cryptography)
├── Doxyfile          – Doxygen configuration
├── docs/
│   ├── report.pdf    – This report
│   └── doxygen/      – Generated Doxygen documentation
│       └── html/
│           └── index.html
├── ttp.log           – TTP server log (generated at runtime)
└── server.log        – Server log (generated at runtime)
```

9. Bibliography

1. **cryptography library documentation** — Python Cryptographic Authority. cryptography (v41.0+). Available at: https://cryptography.io/en/latest/
2. **J. Jonsson and B. Kaliski** — "Public-Key Cryptography Standards (PKCS) #1: RSA Cryptography Specifications Version 2.1". RFC 3447, February 2003.
3. **M. Dworkin** — "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC". NIST SP 800-38D, November 2007.
4. **D. Cooper et al.** — "Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile". RFC 5280, May 2008.
5. **Q. Dang** — "Recommendation for Applications Using Approved Hash Algorithms". NIST SP 800-107, Rev 1, August 2012.

Generated as part of the BSK project submission — Gdańsk University of Technology, June 2026